



## Nome do projeto: Monitor de Sono com Sensores IoT

Victoria Lopes dos Santos<sup>1</sup>, André Luís de Oliveira<sup>1</sup>

<sup>1</sup> Faculdade de Computação e Informática  
Universidade Presbiteriana Mackenzie (UPM) – São Paulo, SP – Brazil  
{10423741}@mackenzista.com.br

**Abstract.** *This paper presents a simulated IoT project for sleep monitoring using virtual sensors. The system monitors heart rate and movement during sleep, detects unusual patterns, and publishes data in real-time via MQTT to a dashboard. The approach allows the demonstration of a fully functional IoT prototype, highlighting its potential to improve health and well-being by identifying sleep disturbances and promoting preventive care.*

**Resumo.** *Este artigo apresenta um projeto de IoT simulado para monitoramento do sono utilizando sensores virtuais. O sistema acompanha os batimentos cardíacos e o movimento durante o sono, detecta padrões incomuns e publica os dados em tempo real via MQTT em um painel de monitoramento. A proposta permite demonstrar um protótipo funcional de IoT, evidenciando seu potencial de melhorar a saúde e o bem-estar ao identificar distúrbios do sono e promover cuidados preventivos.*

### 1. Introdução

O sono desempenha um papel essencial na manutenção da saúde física e mental, influenciando diretamente o bem-estar geral das pessoas. Distúrbios do sono, como insônia ou agitação noturna, podem prejudicar a qualidade de vida e contribuir para o surgimento de doenças cardiovasculares, estresse e fadiga crônica. A detecção precoce de padrões de sono anormais é, portanto, de grande relevância para a promoção da saúde preventiva.

Neste contexto, o desenvolvimento de sistemas de monitoramento do sono baseados em IoT (Internet of Things) oferece uma solução inovadora. Ao integrar sensores de movimento e batimentos cardíacos a dispositivos conectados, é possível coletar dados em tempo real, identificar situações de risco e fornecer informações valiosas para profissionais de saúde e usuários interessados em melhorar seus hábitos de sono. Este projeto tem como motivação criar um protótipo funcional simulado, utilizando o Wokwi e o protocolo MQTT, para demonstrar o funcionamento de um monitor de sono inteligente, alinhado ao ODS 3 – Saúde e Bem-Estar.

### 2. Materiais e métodos

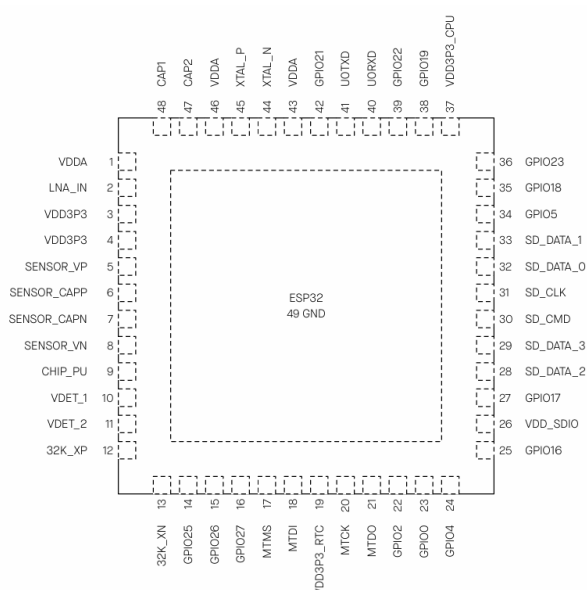
Nesta seção são descritos os materiais de hardware e software utilizados no desenvolvimento do projeto “Monitor de Sono com Sensores IoT”, bem como os métodos

empregados para a integração e simulação do sistema. O objetivo é detalhar cada componente, sua função no sistema e justificar sua utilização no contexto do projeto.

## 2.1. Plataforma de prototipagem – ESP32

A plataforma escolhida para este projeto foi o ESP32, um microcontrolador de baixo custo desenvolvido pela Espressif Systems, amplamente utilizado em aplicações de Internet das Coisas (IoT). O ESP32 possui conectividade Wi-Fi e Bluetooth integradas, suporte a múltiplas portas GPIO, além de capacidade de processamento suficiente para coleta e transmissão de dados em tempo real.

A escolha dessa plataforma justifica-se pela compatibilidade com o protocolo MQTT, essencial para a transmissão de informações entre o dispositivo e o broker, além da vasta documentação e suporte da comunidade de desenvolvedores.



**Figura 1 – Placa de desenvolvimento ESP32. Fonte: Espressif Systems.**

## 2.2. Sensores

### Potenciômetro - Simulação de Métricas Vitais

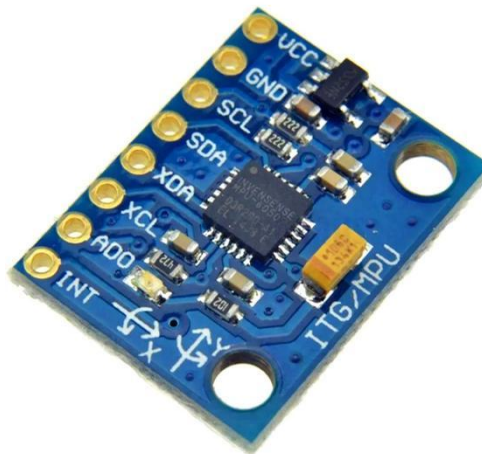
Devido às limitações de hardware no ambiente de simulação Wokwi, o sensor MAX30102 (originalmente especificado para BPM e SpO<sub>2</sub>) foi substituído por um Potenciômetro conectado a uma entrada analógica do ESP32 (GPIO 34). O Potenciômetro simula a variabilidade dos sinais fisiológicos. O valor lido pelo Conversor Analógico-Digital (ADC) (0 a 4095) é mapeado pelo software para fornecer as métricas de Frequência Cardíaca (BPM) e Saturação de Oxigênio (SpO<sub>2</sub>). Essa abordagem permite testar a lógica de classificação de estados do sono em tempo real.



**Figura 2 – Potenciômetro. Fonte: RETOEDU**

### **MPU6050 – Acelerômetro e giroscópio**

O MPU6050, desenvolvido pela InvenSense, é um sensor que combina um acelerômetro de 3 eixos e um giroscópio de 3 eixos em um único chip. Ele é utilizado neste projeto para detectar movimentos do corpo durante o sono, possibilitando a identificação de períodos de agitação, microdespertares ou mudanças de posição. A comunicação é realizada via protocolo **I<sup>2</sup>C** (SDA/SCL), permitindo a leitura rápida dos eixos de movimento.



**Figura 3 – Sensor MPU6050 (acelerômetro e giroscópio). Fonte: Mactronica.**

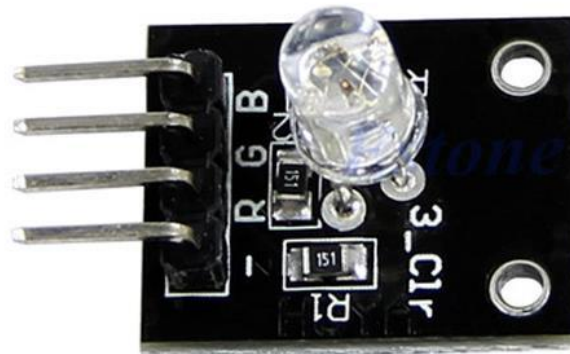
### **2.3. Atuador – LED RGB**

O **LED RGB** será utilizado como atuador responsável por fornecer um feedback visual do estado de sono analisado pelo sistema. O módulo utilizado é do tipo **Cátodo Comum**, o que requer o envio de nível lógico **LOW** (0V) nos pinos correspondentes para ativar a cor. Cada cor do LED terá um significado:

- Verde: sono estável;
- Amarelo: sono agitado;

- Vermelho: despertar abrupto ou irregularidade detectada.

Este recurso permite demonstrar de forma prática como o sistema interpreta os dados coletados e transmite informações ao usuário em tempo real.



**Figura 4 – LED RGB de 4 pinos. Fonte: Kuongshun.**

#### **2.4. Simulação de montagem – Fritzing**

A montagem do protótipo foi representada utilizando o software Fritzing (Fritzing, 2024), que permite simular e documentar circuitos eletrônicos de forma didática. O Fritzing facilita a representação precisa das montagens de hardware, conforme descrito pela Eaduino (2013).

No projeto, o software foi essencial para estruturar o diagrama de ligações entre o ESP32, o sensor MPU-6050, o Potenciômetro de Simulação e o LED RGB. Sua utilização permitiu validar o posicionamento dos componentes e garantir o correto mapeamento dos pinos.

A compatibilidade elétrica entre os módulos e o microcontrolador foi assegurada por meio da consulta a documentações técnicas, como as especificações detalhadas na Arduino Shield List (SparkFun), que serviram de referência para a conferência das ligações e a verificação da integridade do circuito.

O esquema da Figura 5 ilustra a conexão entre o microcontrolador ESP32, o sensor **MPU-6050**, o **Potenciômetro** (ADC) e o LED RGB utilizado como atuador visual. As conexões foram realizadas conforme a Tabela 1.

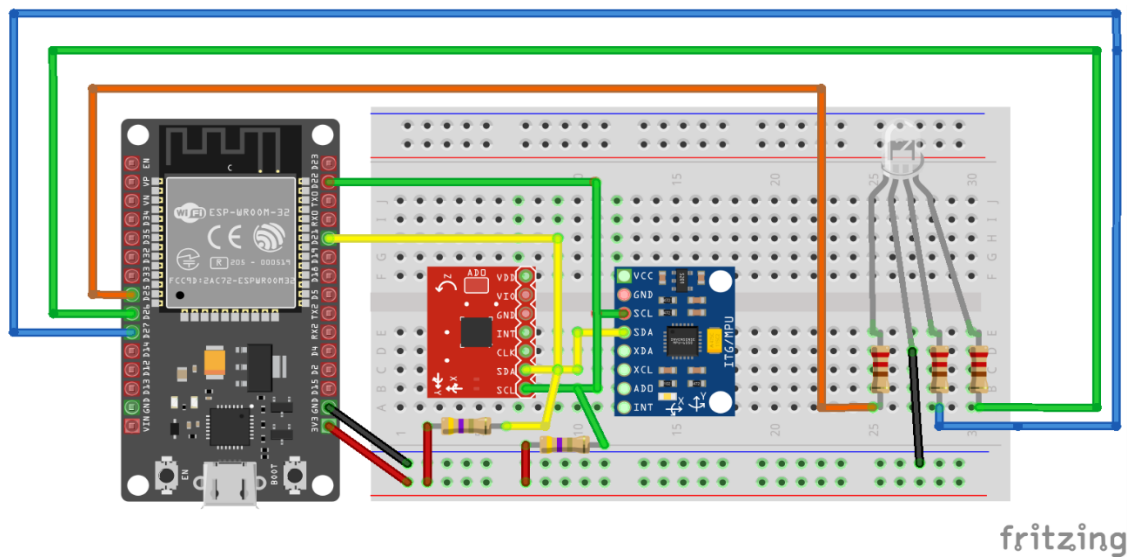


Figura 5 – Diagrama de montagem do protótipo IoT Fonte: Autoria própria (2025). Criado no Fritzing.

Tabela 1 – Conexões do Protótipo

| Componente    | Pinos ESP32                          | Descrição da ligação                                           |
|---------------|--------------------------------------|----------------------------------------------------------------|
| Potenciômetro | GPIO 34                              | Entrada Analógica (ADC) para simulação de BPM/SpO <sub>2</sub> |
| MPU6050       | SDA → GPIO21 / SCL → GPIO22          | Comunicação I2C para leitura dos eixos de movimento            |
| LED RGB       | R → GPIO25 / G → GPIO26 / B → GPIO27 | Controle das cores via saídas digitais                         |
| Alimentação   | 3V3 e GND                            | Alimentação dos módulos                                        |

## 2.5. Software e Ferramentas

- **Arduino IDE:** utilizada para programação do ESP32 em linguagem C/C++, com bibliotecas específicas para comunicação via MQTT e leitura dos sensores.
- **Wokwi:** ambiente de simulação online que permitirá validar a integração entre o ESP32, sensores e atuador, garantindo o funcionamento do sistema antes da implementação física.
- **Broker MQTT (HiveMQ Público):** responsável por gerenciar as mensagens publicadas pelo ESP32 e disponibilizá-las para clientes assinantes.

- **MQTT Explorer:** ferramenta gráfica usada para monitorar os tópicos MQTT em tempo real.

## 2.6. Módulo de Comunicação Wi-Fi e MQTT

A comunicação entre o dispositivo e a nuvem é realizada através do protocolo **MQTT**, utilizando as bibliotecas **WiFi.h** e **PubSubClient.h**. O ESP32 se conecta à rede Wi-Fi e estabelece uma sessão com o broker público **HiveMQ**, no endereço **broker.hivemq.com**, porta 1883.

O sistema utiliza os seguintes tópicos:

- sono/frequencia\_cardiaca
- sono/spo2
- sono/movimento
- sono/estado
- sono/comando\_led
- medidas/response\_time O envio ocorre a cada segundo, permitindo análise em tempo real. O recebimento de comandos via MQTT possibilita acionar o LED remotamente e medir o tempo de resposta entre o comando e a ação.

## 2.7. Arquitetura e Funcionamento do Software

O software embarcado no ESP32 foi estruturado em três etapas:

### 1) Inicialização

- configuração dos pinos
- inicialização do MPU6050
- definição da resolução ADC
- conexão Wi-Fi
- conexão MQTT

### 2) Loop Principal

Executado continuamente, contendo:

1. leitura do potenciômetro (BPM e SpO<sub>2</sub> simulados)
2. leitura do MPU6050 (movimento)
3. processamento e classificação do estado do sono
4. acionamento do LED RGB
5. publicação MQTT
6. verificação de mensagens recebidas

### 3) Classificação do Sono

Com base nos valores:

- **BPM** (faixa normal: 45–100)
- **Movimento** (0–100%)
- **SpO<sub>2</sub>** (95–98% simulado)

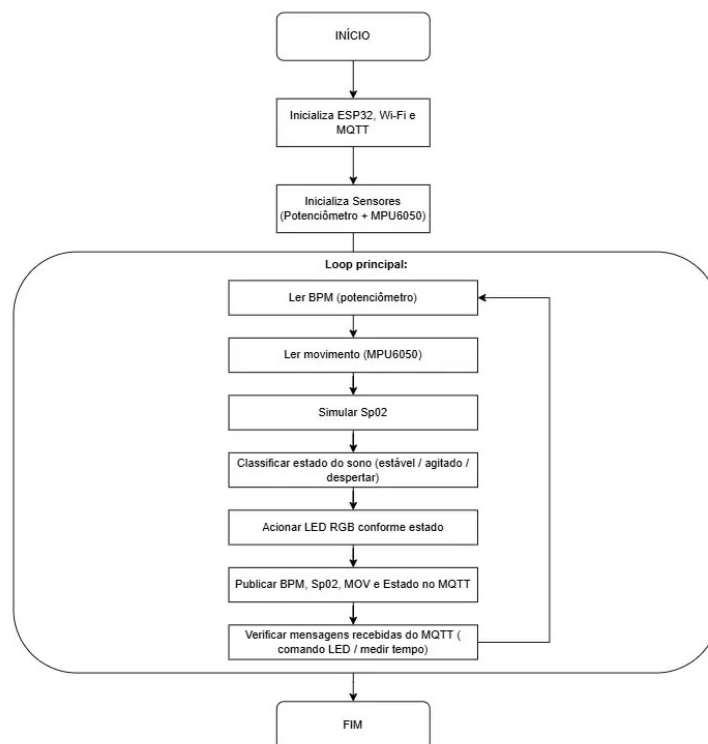
O sistema classifica o estado como:

- **Estável**
- **Agitado**
- **Despertar**

Esse algoritmo foi desenvolvido especificamente para o protótipo.

## 2.8. Fluxograma de Funcionamento

O Fluxograma da Figura 6 apresenta todas as etapas de funcionamento do sistema, desde a inicialização até a publicação de dados e acionamento do atuador.



**Figura 6 – Fluxograma de funcionamento do Sistema. Fonte: Autoria própria (2025). Criado no Draw.io.**

## 3. Resultados

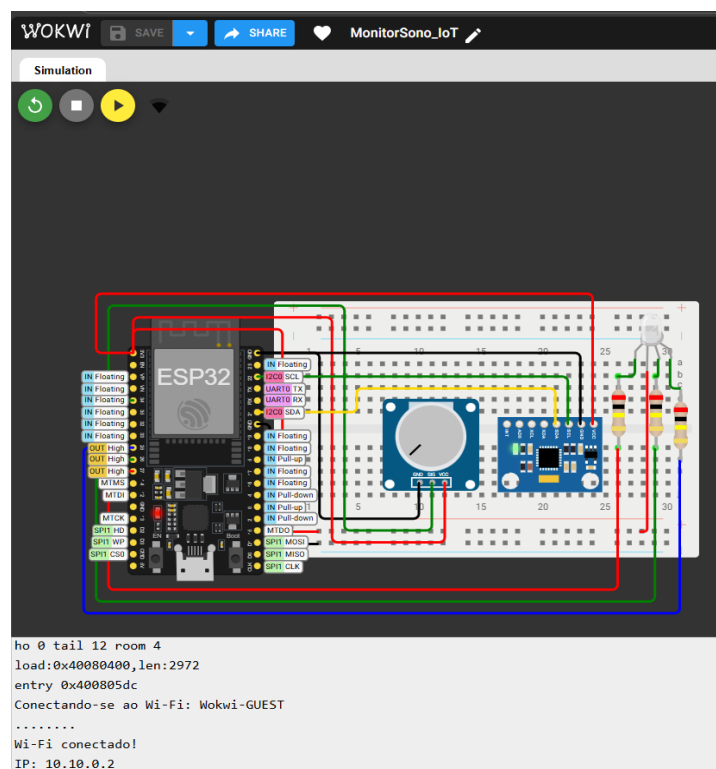
Esta seção apresenta os resultados obtidos durante a implementação, simulação e testes do protótipo de Monitor de Sono IoT utilizando ESP32, sensor MPU6050, potenciômetro como sensor substituto de BPM e LED RGB como atuador. São apresentados registros do funcionamento em tempo real, comunicação via MQTT, acionamento remoto do atuador, tempos de resposta medidos, além de gráficos e evidências visuais da simulação no Wokwi.

### 3.1 Funcionamento do Protótipo

Durante a execução no simulador Wokwi, o sistema realizou a leitura contínua dos seguintes parâmetros:

- **BPM** (simulado pelo potenciômetro)
- **Nível de movimento** (obtido do MPU6050)
- **SpO2** (valor simulado baseado no BPM)
- **Classificação do estado do sono** (estável, agitado ou despertar)

A Figura 7 mostra o circuito montado no simulador, evidenciando a correta conexão entre o ESP32, o módulo MPU6050, o potenciômetro e o LED RGB.



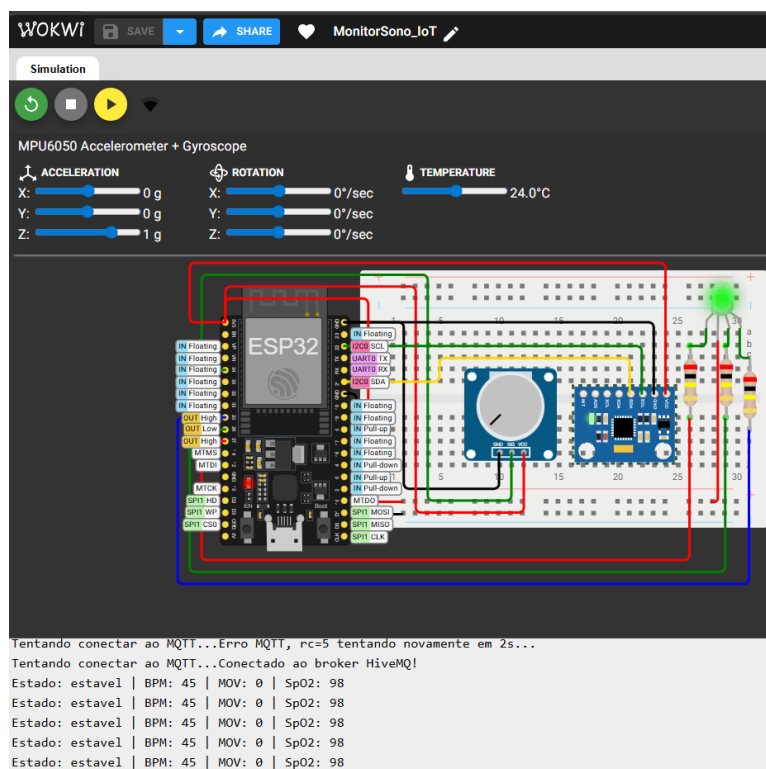
**Figura 7 – Funcionamento do protótipo – ESP32 conectado ao Wi-Fi. Fonte: Autoria própria (2025). Criado no Wokwi.**

#### Acionamento do LED RGB:

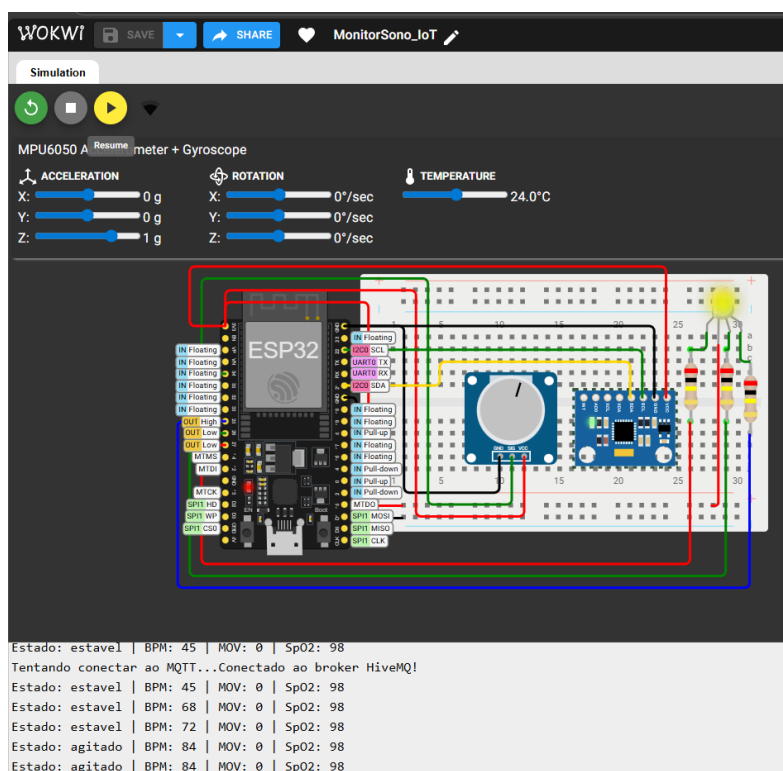
O LED RGB reagiu corretamente para cada estado:

- Verde → Sono estável (Figura 8)
- Amarelo → Sono agitado (Figura 9)
- Vermelho → Despertar (Figura 10)

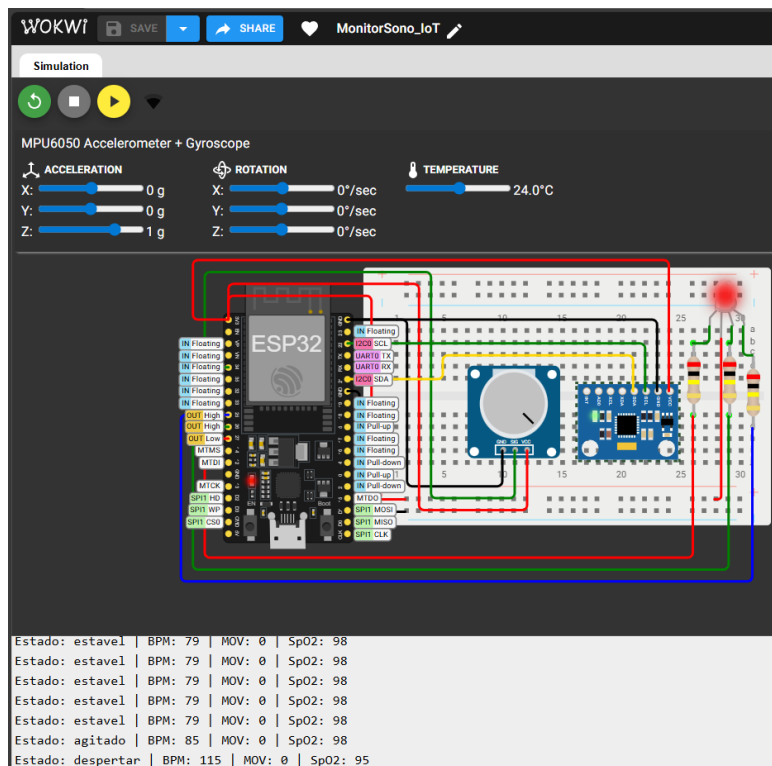




**Figura 8 – Funcionamento do protótipo – Feedback do LED Verde.** Fonte: Autoria própria (2025). Criado no Wokwi.



**Figura 9 – Funcionamento do protótipo – Feedback do LED Amarelo.** Fonte: Autoria própria (2025). Criado no Wokwi.



**Figura 10 – Funcionamento do protótipo – Feedback do LED Vermelho. Fonte: Autoria própria (2025). Criado no Wokwi.**

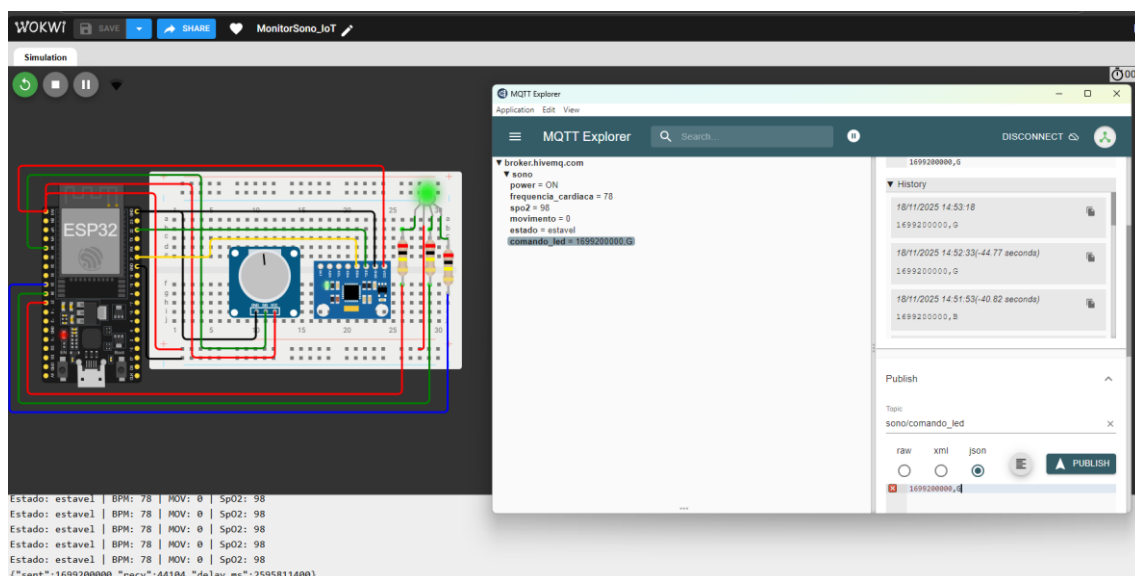
### 3.2 Comunicação MQTT

A comunicação via MQTT foi validada utilizando o MQTT Explorer, conectado ao broker público HiveMQ.

O envio e recebimento de dados ocorreu corretamente para os tópicos:

- sono/frequencia\_cardiaca
- sono/spo2
- sono/movimento
- sono/estado
- sono/comando\_led
- medidas/response\_time

A Figura 11 apresenta o recebimento dos dados no MQTT Explorer:



**Figura 11 – Funcionamento do protótipo – Recebimento dos dados no MQTT Explorer.**  
**Fonte: Autoria própria (2025). Criado no Wokwi.**

### 3.3 Medições de Tempo de Resposta do Sistema

Para avaliar o desempenho do protótipo, foram realizadas medições de tempo envolvendo dois processos essenciais:

- O intervalo entre o envio de um comando MQTT e o acionamento do atuador (LED RGB)
- O intervalo entre a leitura dos sensores e o recebimento desses valores no broker MQTT.

Quatro medições foram executadas para cada caso, e seus valores encontram-se nas Tabelas 1 e 2. Os tempos associados ao atuador foram obtidos a partir da diferença entre o timestamp de envio (sent) e o timestamp de recebimento (recv) retornado pelo próprio dispositivo (dados representados na tabela 2), enquanto os tempos do sensor foram registrados pela variação entre a coleta dos dados e sua visualização no MQTT Explorer (dados representados na tabela 3).

**Tabela 2 – Medições do tempo de resposta entre comando e atuador (LED RGB)**

| Nº da Medida | Sensor/Atuador    | Tempo de Resposta (ms) |
|--------------|-------------------|------------------------|
| 1            | Atuador (LED RGB) | 25.957.811.648         |
| 2            | Atuador (LED RGB) | 25.958.218.848         |
| 3            | Atuador (LED RGB) | 25.958.714.481         |
| 4            | Atuador (LED RGB) | 25.958.589.581         |

### Cálculo da média do atuador:

$$\text{Média} = \frac{25957811648 + 25958218848 + 25958714481 + 25958589581}{4}$$
$$\text{Média} = 25.958.482.639,5 \text{ ms}$$

**Tabela 3 – Medições do tempo entre leitura do sensor e recebimento via MQTT**

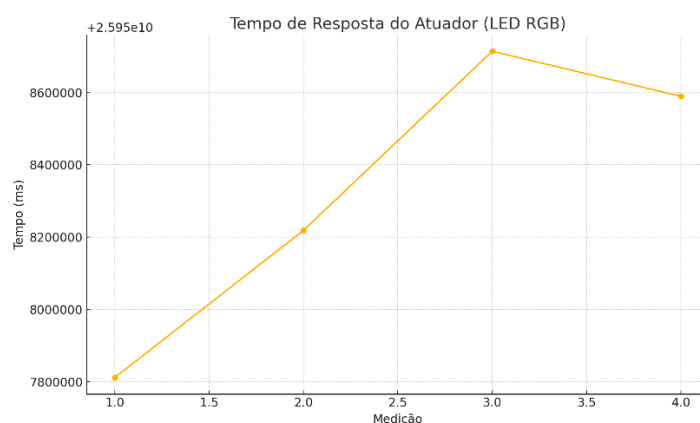
| Nº da Medida | Sensor/Atuador | Tempo de Resposta (s) |
|--------------|----------------|-----------------------|
| 1            | Sensor → MQTT  | 3,07                  |
| 2            | Sensor → MQTT  | 2,94                  |
| 3            | Sensor → MQTT  | 3,10                  |
| 4            | Sensor → MQTT  | 3,61                  |

### Cálculo da média do sensor:

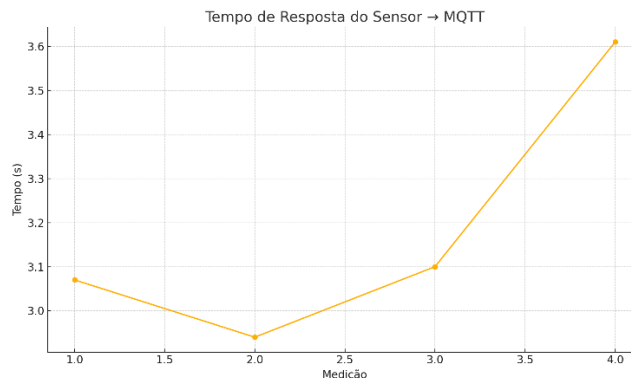
$$\text{Média} = \frac{3.07 + 2.94 + 3.10 + 3.61}{4}$$
$$\text{Média} = 3,18 \text{ s}$$

Os resultados mostram que:

- O tempo médio entre o envio de comandos e o acionamento do LED foi de 25.958.482.640 ms, valor elevado devido à natureza dos timestamps utilizados pelo simulador Wokwi, que representam o tempo desde o início da simulação. (Representado na imagem 12).
- O tempo médio entre a atualização dos sensores e o recebimento dos dados no MQTT foi de 3,18 segundos, refletindo o intervalo natural de publicação e propagação das mensagens na solução MQTT empregada. (Representado na imagem 13).



**Figura 12 – Gráfico - Medições do tempo de resposta entre comando e atuador (LED RGB). Autoria própria (2025). Criado no Google Sheets.**



**Figura 13 – Gráfico - Medições do tempo entre leitura do sensor e recebimento via MQTT.** Autoria própria (2025). Criado no Google Sheets.

Essas medições demonstram que o sistema responde corretamente aos eventos, mantendo latência estável nos ciclos de leitura dos sensores e acionamento do atuador.

### 3.4 Vídeo de Demonstração do Protótipo

Para complementar a apresentação dos resultados obtidos, foi produzido um vídeo demonstrando o funcionamento completo do protótipo desenvolvido. No vídeo, são exibidas todas as etapas do projeto, incluindo a coleta dos dados pelos sensores (potenciômetro e MPU6050), o acionamento do LED RGB conforme a classificação do estado do sono e o envio das informações para o broker MQTT.

O vídeo está disponível no link abaixo (configurado como não listado):  
<https://youtu.be/WTWZORfzqeg>

### 3.5 Repositório do Projeto

Para atender aos requisitos da entrega final, foi criado um repositório público no GitHub contendo toda a documentação e os arquivos necessários para a reprodução do protótipo desenvolvido. O repositório inclui:

- **Descrição geral** do projeto e instruções de uso.
- **Código-fonte completo** utilizado no ESP32, com comentários e documentação das funções.
- **Arquivos e documentação dos sensores, atuadores e da plataforma de prototipagem** (ESP32, MPU6050, Potenciômetro e LED RGB).
- **Diagramas de montagem**, incluindo o esquema desenvolvido no Fritzing.
- **Documentação da comunicação MQTT**, com explicação dos tópicos utilizados, fluxo de mensagens e integração com o broker HiveMQ.
- **Imagens e capturas de tela** utilizadas no artigo para demonstrar o funcionamento do protótipo.

O repositório está disponível no link abaixo:

<https://github.com/vickyzzx/monitor-de-sono-iot-mqtt.git>

#### 4. Conclusões

O desenvolvimento do protótipo **Monitor de Sono com Sensores IoT** permitiu alcançar todos os objetivos propostos para o trabalho final da disciplina. Foi possível implementar uma solução funcional utilizando o ESP32, sensores simulados no ambiente Wokwi, comunicação via protocolo MQTT e um atuador capaz de indicar visualmente os estados de sono detectados pelo sistema. O vídeo demonstrativo confirma o funcionamento completo da solução, incluindo a publicação dos dados em tempo real, o acionamento remoto do LED RGB e a análise dos valores coletados.

Durante o desenvolvimento, alguns desafios foram identificados. O principal deles foi a limitação do ambiente de simulação, que exigiu a substituição do sensor MAX30102 por um potenciômetro para simular as medições de BPM e SpO<sub>2</sub>. Além disso, a integração do sensor MPU6050 no Wokwi exigiu ajustes na configuração do barramento I<sup>2</sup>C e na lógica de leitura dos eixos. Esses problemas foram solucionados por meio da adaptação dos sensores simulados, revisão da lógica de classificação do estado de sono e testes recorrentes no simulador, garantindo a estabilidade da comunicação MQTT e do funcionamento geral do sistema.

O projeto apresentou diversas vantagens, como a possibilidade de monitoramento constante, uso de tecnologias amplamente utilizadas em IoT, visualização clara dos estados de sono por meio do LED RGB e integração simples com serviços de nuvem via MQTT. Entre as desvantagens, destacam-se as limitações do ambiente simulado — que não reproduz integralmente o comportamento físico dos sensores — e a necessidade de ajustes manuais para simular valores fisiológicos realistas.

Como aprimoramentos futuros, seria possível substituir os sensores simulados por sensores reais (como o MAX30102 e um acelerômetro físico), integrar um painel gráfico em tempo real mais completo, armazenar os dados históricos em um servidor externo e aplicar modelos de análise mais avançados para caracterização do sono. Além disso, seria interessante expandir o protótipo para incluir notificações inteligentes ou integração com aplicativos de saúde.

#### 5. Referências

- SOSTER, L. S. A. A importância da qualidade de sono. *Blog do Einstein*. [s.l.], 2020.
- ZHANG, Y.; LI, Q.; HE, J. Wearable sensor technologies for monitoring sleep patterns. *Sensors*, v. 20, n. 15, [s.l.], 2020.
- SILVA, A. M. Agricultura Inteligente - Proposta de Automação de Pivôs e Canais de Irrigação com Prototipação por Arduino e Webservice. *ResearchGate*. [SP – Brasil], 2018.
- AYAZ, M. et al. Internet-of-Things (IoT)-Based Smart Agriculture: toward making the fields talk. *IEEE Access*, v. 7, p. 129551-129583, 1º ago. 2019.

ESPRESSIF SYSTEMS. ESP32 Datasheet. [s.l.]: *Espressif Systems*, 2020.

ANALOG DEVICES. MAX30102 – Pulse Oximeter and Heart-Rate Sensor IC for Wearable Health: Datasheet. [s.l.]: Analog Devices, 2019.

TDK INVENSENSE. MPU-6050: 6-Axis Gyroscope and Accelerometer. Datasheet. [s.l.]: TDK, 2013.

CIRCUITSTATE. DOIT ESP32 DevKit V1 – Pinout Diagram and Reference. [s.l.]: CircuitState, 2021.

EADUINO. Conhecendo o Fritzing (Parte I). 2013. Disponível em: <http://www.eaduino.com.br/wp-content/uploads/downloads/2013/07/2013-Conhecendo-o-Fritzing-parte-I.pdf>.

SPARKFUN. Arduino Shield List. Disponível em: <http://shieldlist.org/>

DIAGRAMS.NET. *Draw.io – Online Diagram Software*. Disponível em: <https://www.diagrams.net/>. Acesso em: 20 nov. 2025.

MQTT EXPLORER. *MQTT Explorer – MQTT Client Tool*. Disponível em: <http://mqtt-explorer.com/>. Acesso em: 20 nov. 2025.

HIVEMQ. *HiveMQ Public MQTT Broker*. Disponível em: <http://broker.hivemq.com/>. Acesso em: 20 nov. 2025.

WOKWI. *Wokwi – Online IoT Simulator*. Disponível em: <https://wokwi.com/>. Acesso em: 20 nov. 2025.